Cross-Step Memory-Lifetime Characterization of LLM Agent-Workflow Replays for Differentiated Memory

Minseok Kim
Stanford University
dkim27@stanford.edu

Kristen Guernsey
Stanford University
kguerns@stanford.edu

***Abstract*—LLM agents execute multi-step workflows in which context accumulates and is repeatedly re-read across steps. Prior data-lifetime profiling (e.g., GainSight) characterizes activations *within a single forward pass*, leaving the *cross-step* memory behavior of agentic inference unquantified. We instrument deterministic agent-workflow replays—coding, search, and context-growth compaction—on an NVIDIA H100 with vLLM and Qwen2.5-Coder-7B-Instruct, logging every `create/read/mutate/free` on every logical object across text, token, and analytically-projected KV representations. We report two findings. First, agent memory is *carry-over dominated*: by the final step, 87–99% of the projected-KV working set is context created in earlier steps, and only compaction (summarize-and-demote) reduces the resident set; prefix caching makes re-reads cheap but frees nothing. Second, capacity-time pressure and reuse intensity are *decoupled* per semantic class—two classes read the same 30 times differ by $4.6\times$ in byte-seconds—so placement must key on the joint (size$\times$lifetime, reuse) rather than lifetime or reuse alone. From these observations we derive a prescriptive three-tier mapping of agent data classes onto a differentiated memory system. We frame this as mechanism-based characterization on a small replay set, not a generalizable benchmark.**

***Index Terms*—differentiated memory, tiered memory, LLM inference, agentic workloads, KV cache, data lifetime, memory characterization**

I. Introduction

Large language model (LLM) *agents* solve tasks as multi-step workflows: they read context, call tools, and append the results to a growing prompt that is re-processed on every step. The memory footprint of this loop—prompt text, token IDs, and the attention key/value (KV) cache—is what a memory system must hold, yet most data-lifetime analysis stops at the granularity of a single forward pass [1]. The question we ask is orthogonal: *how does a logical unit of agent data live and get reused across the steps of a session, and what does that imply for placing it in a differentiated (tiered) memory system (DMS)?*

We make three contributions:

1) A logical-layer tracer and a deterministic replay harness that record the full lifecycle (`create`/`read`/`mutate`/`free`) of every content-identified object, across text, token, and projected-KV representations, for three agent-workflow families on real H100/vLLM hardware (Section III).
2) A measurement that agent memory is *carry-over dominated* (87–99% of the per-step KV working set is re-materialized prior context), with compaction as the only operation that shrinks the resident set (Section V).
3) Evidence that capacity-time pressure and reuse are decoupled per semantic class, yielding a prescriptive (size$\times$lifetime, reuse) tier mapping (Section VI).

We are explicit about scope: the artifact is an experimental *characterization* framework, not a simulator or a benchmark. Results are tightly coupled to one 7B model and a small, scripted replay set, and the tier mapping is prescriptive from logical observations rather than measured physical placement (Section VII).

II. Background and Related Work

Within-pass lifetime profiling. GainSight [1] profiles data lifetime at activation / within-forward-pass granularity. It motivates our methodology but does not capture behavior that spans agent steps, which is where agentic context accumulation lives.

Agent memory frameworks. ReCA [2] proposes a dual (short- vs. long-term) memory organization for agentic systems, and DualPath [3] shows multi-turn agent inference can become memory- or I/O-bound. These motivate a tiered treatment of agent state; we contribute the per-class, cross-step measurements that such a policy would key on.

LLM serving and the KV cache. vLLM [4] manages KV in paged blocks and supports prefix caching, which we exploit and instrument. KV grows linearly with context length, which makes its capacity behavior the dominant first-order memory effect in multi-step prompts.

III. Methodology

Figure 1 summarizes the pipeline: scripted workload fixtures feed a deterministic replay runner, three instrumentation streams emit events, validation gates guard correctness, and analysis modules produce the metrics and the tier mapping.

A. Logical-layer tracer

The tracer emits one JSON event per memory operation with fields {`ts`, `step`, `phase`, `object_id`, `logical_id`, `repr_type`, `size_bytes`, `op`,

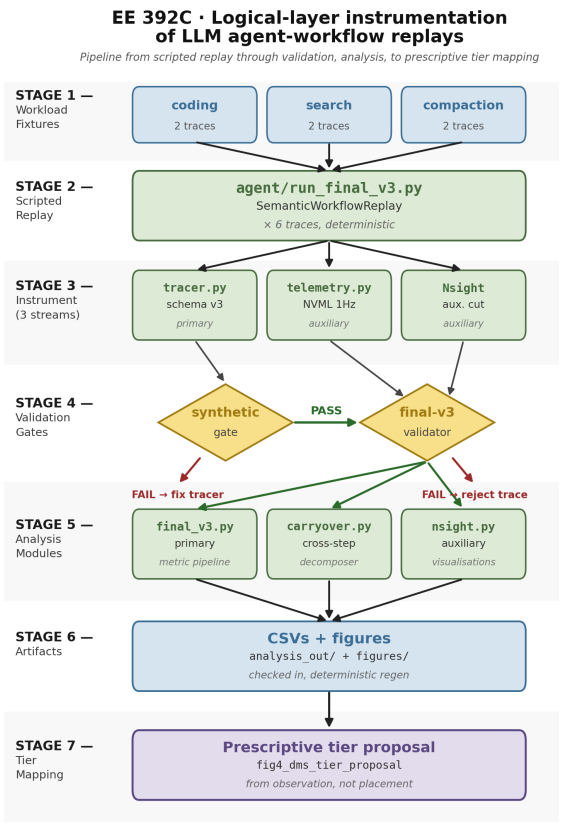

Fig. 1. Instrumentation-to-analysis pipeline. Tool *choices* are scripted so the memory shape is repeatable; the LLM *text* is genuinely generated and fed back into history.

`semantic_type, ...`}. Two design choices make the cross-step analysis possible:

- **Content identity (`logical_id`).** A normalized content hash: the same content reused later shares an id, while an edit yields a new id (a new version). This lets us track a logical object across steps independent of its physical buffer.
- **Representations (`repr_type`).** The same logical object is recorded as text, token-IDs, and projected KV, so we can separate representation overhead from logical content.

A `semantic_type` tags each object with its application class (`system_prompt`, `raw_context`, `search_result`, ...), the axis along which we group all metrics.

B. KV estimation

KV bytes are an analytical, GQA-aware projection from the model configuration:

$$\text{kv\_bytes/token} = 2 \cdot L \cdot H_{kv} \cdot d_{head} \cdot b = 2 \cdot 28 \cdot 4 \cdot 128 \cdot 2 = 57{,}344, \quad (1)$$

which scales linearly with token count. Per-step KV spans are bounded at the *next prefill boundary*, not task end, so projected KV is never presented as permanent residency. Cache-adjusted KV uses vLLM's `RequestOutput.num_cached_tokens`; this is an availability and count-reconciliation check, not independent semantic-attribution ground truth.

TABLE I
THE SIX FINAL-V3 TRACES (3 FAMILIES × 2 CONDITIONS).

Family	Steps	Contrast (independent variable)
`coding_agent`	5	prefix caching on vs. off
`search_agent`	4	targeted vs. broad retrieval
`compaction_agent`	5	compaction on vs. off

C. Workload replays

Each workload is a deterministic, scripted replay run in two contrast conditions (Table I). Tool choices are fixed for repeatability, but the model output is generated by vLLM and appended to history, preserving agent-like prompt/tool structure. Search and compaction expand small seed fixtures at runtime so the prompt is large enough to stress the intended behavior without committing bulky files.

D. Metrics

Per semantic class we compute: **lifetime** (create → last access, reported step-normalized and in seconds); **reuse** (logical read events); **byte-seconds** (size × lifetime, i.e. capacity-time pressure); **KV pressure** (logical / cached-reuse / cache-adjusted-new); **duplication** (bytes held vs. unique logical bytes); and **cross-step carry-over** (per-step KV split into new-prefill vs. carried-from-earlier).

E. Validation

A hand-built synthetic trace with known-correct expectations gates tracer correctness; a final-v3 validator gates the real traces (schema, span contiguity, `size_bytes` = `tokens` × 57,344, cached-token availability, mutate⇒new `logical_id`). Both pass for all six traces.

IV. Experimental Setup

All traces run on a RunPod NVIDIA H100 80 GB HBM3 with vLLM 0.10.2 (in-process `LLM`) and Qwen2.5-Coder-7B-Instruct [5], bf16, `max_model_len`=8192, temperature 0, seed 42. Prompts grow to ∼6.3 K tokens; each task is capped at 15 steps. We collect six traces and report mechanism contrasts—there is no within-condition replication, so we use no confidence intervals or significance tests.

V. Results

Our results quantify cross-step memory behavior along two dimensions: *lifetime and carry-over* (Subsections V-A–V-D) and *capacity-time vs. reuse* (Subsection V-F). The first reveals that agent state is dominated by re-read context from earlier steps, and only demotions (compaction) shrink the resident set. The second shows that lifetime and reuse do not track each other per semantic class, motivating the joint (size×lifetime, reuse) tier mapping in Section VI.

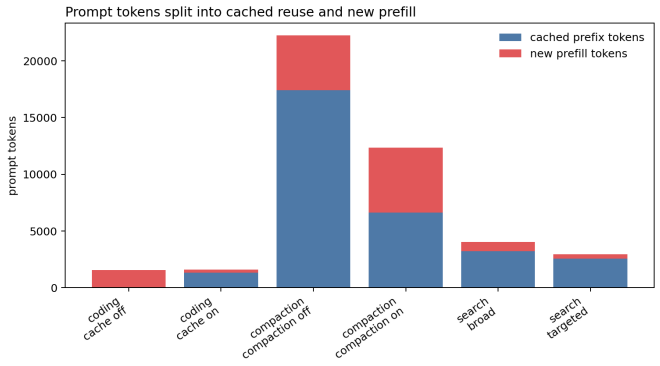

Fig. 2. Prompt tokens split into cached-prefix reuse and new prefill. The coding pair isolates cache policy; search/compaction rows show prompt-shape effects with caching on.

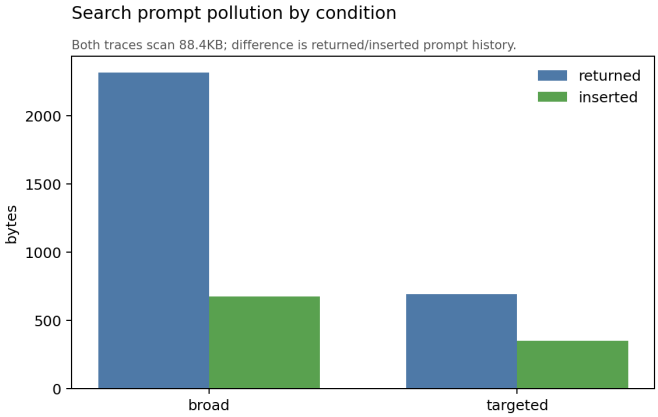

Fig. 3. Returned vs. inserted bytes by retrieval breadth after an identical corpus scan.

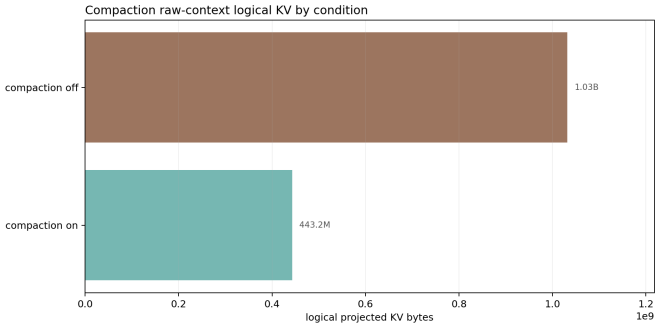

Fig. 4. Retained raw-context logical KV, compaction on vs. off.

A. *Prefix caching converts repeated context into cheap reuse*

In the coding replay, cache-on and cache-off have nearly identical total prompt tokens (1,590 vs. 1,571), but with prefix caching vLLM reports 1,344 cached tokens and only 246 new prefill tokens (Fig. 2). Cache-adjusted new KV falls from 90,087,424 B to 14,106,624 B, an 84.3% reduction; cache-off has zero KV-reuse events. This distinction matters because prefix caching changes *bandwidth work* without changing the logical live set. The same prior context still exists in the prompt and must remain addressable by the serving system; caching only avoids recomputing KV for the prefix that vLLM can recognize. Thus, prefix caching is a reuse optimization rather than a capacity-management policy: it reduces repeated prefill traffic but does not decide which logical objects should remain resident, be demoted, or be summarized.

B. *Retrieval selectivity, not scan volume, drives pollution*

Both search traces scan the same corpus (88,372 B). Broad retrieval returns 2,318 B / inserts 672 B of snippets vs. targeted's 691 B / 350 B; broad `search_result` logical KV is 3.22× targeted (Fig. 3). The scan-volume proxy is excluded from live-object summaries. The important mechanism is that scanning and retaining are different memory events. A broad retrieval policy may read the same external corpus as a targeted policy, but only the returned and inserted snippets become part of the agent's future prompt state. Once inserted, those snippets are repeatedly re-read in later steps and are projected into KV at each prefill boundary. Therefore, retrieval breadth affects DMS pressure through *prompt pollution*: extra context that is not necessarily useful enough to justify its repeated capacity-time cost.

C. *Compaction trades capacity for reuse*

Both compaction traces ingest 18,602 B of raw context. Compaction adds a 513 B summary, demotes (`op=free`) the earlier raw context, and cuts raw-context logical KV from 1.032 GB to 443 MB (Fig. 4). The tradeoff: compaction-on has fewer prompt tokens (12,364 vs. 22,234) but *more* new prefill (5,740 vs. 4,826), because inserting a summary disrupts the long cached prefix. This result shows that compaction is not simply "always better" for every memory metric. It is beneficial for long-term capacity because it frees bulky raw context and replaces it with a much smaller summary, but it can temporarily increase new-prefill work by changing the exact prefix seen by the cache. In DMS terms, compaction is a demotion mechanism: it converts large, low-density context into a smaller representation that is easier to keep in a faster or more resident tier, at the cost of some recomputation when the prompt structure changes.

D. *Agent memory is carry-over dominated*

At every generate step the runner re-reads and re-projects every still-active object, so the per-step KV working set splits into newly-prefilled vs. carried-from-earlier bytes (Fig. 5). By the final step of every default trace, **87–99% of the projected-KV working set originates in an earlier step**. Compaction is the only mechanism that resets this: compaction-on drops to 46% carried at the demotion step while every other trace stays above 90%. Prefix caching makes the re-reads cheap but does not reduce the resident set; only `op=free` does. Carry-over dominance is the central reason agentic inference differs from a single isolated prompt. In a one-shot request, most prefill state is created once for that request; in an agent workflow, earlier context becomes a persistent input to many later steps.

The memory system therefore sees a growing population of objects whose value is not that they are newly created, but that they remain eligible for repeated reuse. This favors policies that explicitly distinguish new step-local data from older carried context, because the latter dominates capacity even when caching reduces recomputation.

E. Semantic classes split into carry-over and single-step cohorts

The carry-over dominance emerges from heterogeneous class lifetimes. We classify objects as *single-step* (created and freed within one generate step) or *multi-step* (created in an earlier step, persisting forward). Single-step data includes new tool results and intermediate generations; multi-step includes all scaffolding and context from prior steps. In the three default traces: coding-cache-on, search-targeted, and compaction-on, single-step KV averages $23{,}245\,\mathrm{B}$ per step (ranging $8{,}172$–$31{,}084\,\mathrm{B}$), while multi-step KV averages $1{,}814{,}229\,\mathrm{B}$ per step—a $78\times$ **capacity dominance**. Single-step objects are typically small (tool results, prompt headers), so the capacity-time pressure concentrates in multi-step objects: raw context, search results, and assistant history. This decomposition explains why a tier mapping cannot rest on lifetime or reuse alone: a class like `system_prompt` may be among the longest-lived (steps 0–4) but consists of tiny objects, while `raw_context` is similarly long-lived but bulky. The next subsection quantifies this decoupling. A useful way to interpret this split is that single-step objects behave like temporary activations of the agent loop, while multi-step objects behave like session state. Temporary objects may need low latency during the current step, but they disappear quickly and contribute little byte-seconds. Session-state objects may be individually less latency-critical, but their repeated survival across steps makes them the primary target for tiering, demotion, or summarization.

F. Capacity-time and reuse are decoupled

Table II gives the per-class signature. `raw_context` and `assistant_history` both take 30 logical reads, but `raw_context` carries $4.6\times$ the byte-seconds; `system_prompt` holds $124\times$ less capacity-time than `raw_context` at comparable reuse. Neither size, lifetime, nor reuse *alone* separates the classes. The implication is that a one-dimensional heuristic can make the wrong placement decision. A reuse-only policy would treat `raw_context` and `assistant_history` similarly because both are read 30 times, even though `raw_context` consumes far more capacity-time. A lifetime-only policy would over-prioritize small persistent scaffolding such as `system_prompt`, even though its footprint is negligible. The joint metric separates these cases: hot and small objects should stay close to the compute path, while bulky persistent objects should be candidates for cheaper capacity tiers or semantic compaction.

Figure 6 visualizes this across all six traces' semantic classes in joint (capacity-time, reuse) space. The y-axis (logical read events) shows reuse intensity; the x-axis (byte-seconds, log scale) shows size×lifetime pressure. The scatter reveals three natural clusters: the smallest, hottest objects (plan state, system prompt) occupy the bottom-left; a middle cohort of active context (assistant history, file contents) spans mid-range byte-seconds with 13–30 reads; and the bulky classes (raw context, broad search results) occupy the top-right, driven by capacity-time despite similar reuse to the middle tier. This empirically motivates assigning tiers on the *joint* metric: a placement policy that keys on lifetime or reuse alone would misplace `raw_context` (high reuse $\Rightarrow$ T1?) and `system_prompt` (long lifetime $\Rightarrow$ T3?), contrary to observations.

TABLE II
PER-CLASS CAPACITY-TIME VS. REUSE (`COMPACTION_OFF`; `COMPACTED_SUMMARY` FROM `COMPACTION_ON`). CAPACITY-TIME AND REUSE DO NOT TRACK EACH OTHER.

Semantic class	Byte-seconds	Reads	Tier
`plan_state`	2.8×10^6	2	T1
`system_prompt`	2.3×10^7	14	T1
`compacted_summary`	6.1×10^7	8	T1
`assistant_history`	6.1×10^8	30	T2
`raw_context`	2.8×10^9	30	T3

We use this plot as a policy-design aid rather than a statistical clustering claim. The goal is not to prove universal cluster boundaries from six traces, but to show that the semantic classes occupy qualitatively different regions of the capacity-time/reuse space. This is enough to motivate the design rule used in the next section: tier decisions should combine semantic role with measured capacity-time pressure, rather than relying on token count, lifetime, or access count in isolation.

VI. ANALYSIS: PRESCRIPTIVE TIER MAPPING

The decoupling in Section V-F means a placement policy must key on the *joint* (size×lifetime, reuse), not lifetime or reuse alone. Figure 7 states the resulting prescriptive mapping:

- **Tier 1 (resident, low-latency):** small, high-reuse scaffolding (`system_prompt`, `plan_state`, `compacted_summary`).
- **Tier 2 (bandwidth):** active KV and multi-step context (`assistant_history`, `file_content`, `tool_result`).
- **Tier 3 (capacity, cheap):** bulky, low-reuse retained context (`raw_context`, broad `search_result`).

The mapping should be read as a memory-management prescription for an agent runtime. Tier 1 is reserved for objects whose small size makes residency cheap and whose repeated use makes latency important. Tier 2 holds active working state that is still frequently referenced but large enough that unlimited replication would be wasteful. Tier 3 absorbs bulky retained context whose main cost is capacity-time; these objects are the natural targets for compression, summarization, eviction, or slower-memory placement.

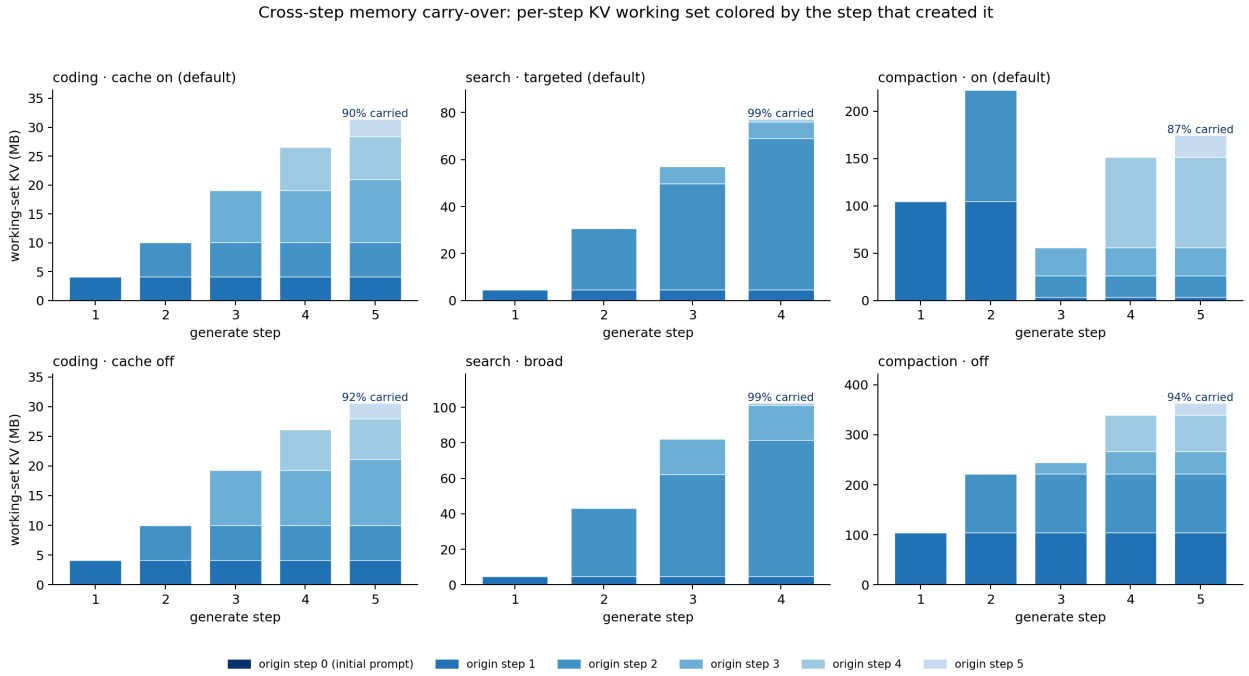

Fig. 5. Per-step KV working set colored by the step that first prefilled each object. Bottom-up strata are carry-over; only the compaction-on panel (top right) resets at the demotion step.

The traces do not measure physical placement; this mapping is prescriptive from logical observations. A practical corollary: *lifetime alone is a poor placement key*—`system_prompt` is among the longest-lived objects yet belongs in the fastest tier because it is small and hot.

Trade-off: semantic knowledge vs. generalization. Within these workloads, semantic classes form the clusters shown in Figure 6. However, generalizing to new workloads raises a question: should tier assignment use semantic class labels (as we do here, incurring no profiling cost), or apply data-driven byte-seconds thresholds? Figure 6 compares the two approaches on the same data: left panel colors by semantic class (our choice), right panel by pure byte-seconds ($<$100 MB·s $\Rightarrow$ T1, 100 MB–1 GB·s $\Rightarrow$ T2, $>$1 GB·s $\Rightarrow$ T3). The byte-seconds approach yields tighter visual clustering and would generalize to any workload after profiling. However, profiling incurs the cost of full-trace collection to compute lifetime; for small or embedded workloads, that overhead may exceed the benefit. Our choice of semantic hardcoding prioritizes *low profiling overhead* at the cost of domain dependence; a scaling strategy would require either accepting semantic input per domain or investing in light-weight byte-seconds profiling.

A practical implementation could combine the two approaches. Semantic labels can provide a cold-start policy before enough trace history exists, while online counters gradually estimate byte-seconds and reuse for each class. The runtime could then override the semantic default when the observed behavior disagrees with the expected class behavior; for example, a normally small `search_result` class could be demoted if broad retrieval causes it to accumulate large byte-seconds.

VII. Limitations

Analytical KV. KV bytes are projected, not measured; we report no hardware memory counters (RunPod containers deny Nsight Compute access to GPU performance counters). **Scripted replays.** Tool choices are deterministic, so we do not claim autonomous-agent generality. **Small set.** Six traces, one 7B model, no within-condition replication—no CIs or significance. **Within-session only.** No cross-session (truly long-term) objects appear, so the long-term tier is proposed, not measured. **Prescriptive placement.** We recommend, but do not measure, physical tier assignment or migration.

These limitations mean that the strongest claims in this paper are comparative and mechanistic: caching reduces repeated prefill, broad retrieval increases retained prompt state, and compaction is the only observed mechanism that frees bulky carried context. We do not claim that the exact thresholds or percentages will hold across models, hardware platforms, or autonomous agents with different tool policies. Instead, the portable result is the measurement framework and the placement principle: agent memory should be characterized by semantic class, capacity-time, and reuse jointly.

VIII. Future Work

Validate the analytical KV on bare metal. With host-level GPU access (e.g., a bare-metal provider where `NVreg_RestrictProfilingToAdminUsers` can be cleared), Nsight Compute can measure per-kernel HBM/L2 traffic and check it against the 57,344 B/token projection, and

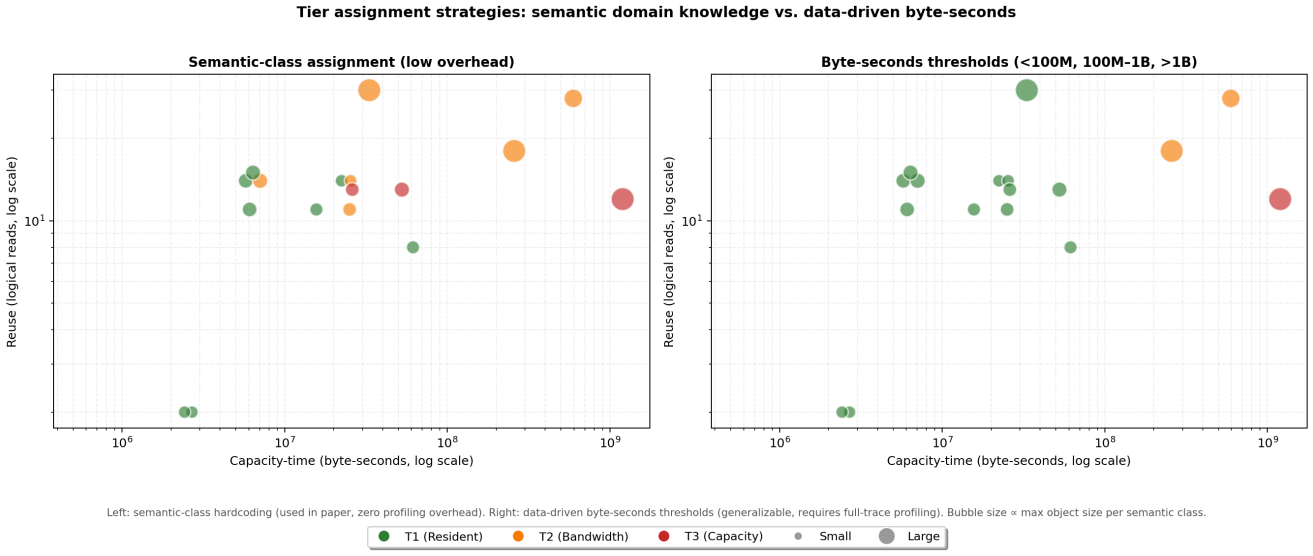

Fig. 6. Joint (capacity-time, reuse) space for all semantic classes across default traces. *Left:* tier assignment by semantic domain knowledge (used in this work, zero profiling overhead). *Right:* tier assignment by data-driven byte-seconds thresholds (more generalizable, requires full-trace profiling). Bubble size $\propto$ max object size per class.

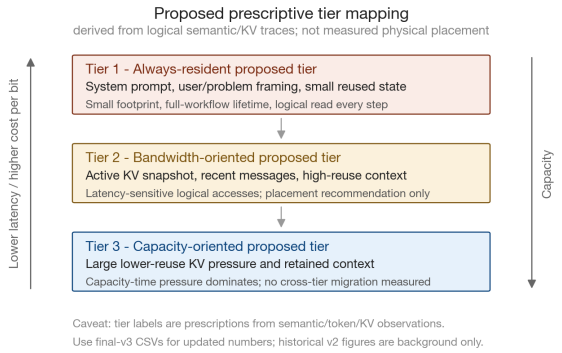

Fig. 7. Prescriptive three-tier mapping derived from logical semantic/token/KV observations (not measured placement).

probe the SRAM/DRAM behavior omitted here. This *complements* the logical layer (it answers a per-kernel, not cross-step, question). **Scale.** Multi-session and longer workflows to exercise a genuine long-term tier; additional models and sizes for variance. **Policy.** A trace-driven simulation of the proposed tier mapping with explicit migration to quantify latency/capacity tradeoffs.

IX. Conclusion

We characterized how LLM agent data lives and is reused *across* workflow steps, not just within a forward pass. Agent memory is carry-over dominated and capacity-time decouples from reuse, so a differentiated memory system should place agent data by the joint (size$\times$lifetime, reuse): small hot scaffolding resident, bulky cold context demoted. We offer this as mechanism-based characterization to guide tiered-memory policy for agentic inference.

Reproducibility and Author Contributions

Code, traces, and figures: https://github.com/ms-d-kim/ee392c-agent-mem. Minseok Kim and Kristen Guernsey jointly defined the project scope, workload families, and evaluation questions. Minseok led the replay harness, vLLM integration, and trace-generation infrastructure. Kristen led the framing of the DMS motivation, interpretation of results, and presentation organization. Both authors contributed to metric design, figure generation, validation, and final writing.

AI Usage

The authors used AI assistance for writing support, including wording suggestions, organization, and editing text. All technical claims, experimental results, figures, code, and interpretations were reviewed and validated by the authors.

References

[1] P. Li, M. Hung, Y. Tan, K. Hoßfeld, J. C. Jiajun, S. Liu, L. Yan, X. Wang, P. Levis, H. S. P. Wong, and T. Tambe, "Gainsight: A unified framework for data lifetime profiling and heterogeneous memory composition," 2025. [Online]. Available: https://arxiv.org/abs/2504.14866

[2] Z. Wan, Y. Du, M. Ibrahim, J. Qian, J. Jabbour, Y. Zhao, T. Krishna, A. Raychowdhury, and V. Reddi, "Reca: Integrated acceleration for real-time and efficient cooperative embodied autonomous agents," 03 2025, pp. 982–997.

[3] Y. Wu, S. Chen, Y. Zhong, R. Huang, Y. Tan, W. Zhang, L. Zhang, S. Zhou, Y. Liu, S. Zhou, M. Zhang, X. Jin, and P. Huang, "Dualpath: Breaking the storage bandwidth bottleneck in agentic llm inference," 2026. [Online]. Available: https://arxiv.org/abs/2602.21548

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[5] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024. [Online]. Available: https://arxiv.org/abs/2409.12186